

Map My Culture

A Comprehensive AI-Powered Cultural Heritage Preservation Platform

DOCUMENT TYPE
Project Report

ORGANISATION
RCTS, IIIT Hyderabad

TABLE OF CONTENTS

No.	Section	Description
01	Executive Summary & Problem Statement	Why this project exists, the pain point, context
02	Project Overview & Objectives	What was built, goals, team scope
03	Proposed Solution & Key Features	How the problem is solved, feature matrix
04	High-Level System Architecture	Five-layer architecture diagram and description
05	Frontend Architecture & Flowchart	Next.js stack, component tree, data flow
06	Backend Architecture & Flowchart	FastAPI processing pipeline, service layers
07	Database Architecture & ER Diagram	Entity schema, relationships, data model
08	Technology Stack	All technologies with versions and rationale
09	API Documentation	Endpoints, parameters, request/response samples
10	Code Implementation Guide	Folder structure, key modules, code snippets
11	Additional Features	Bonus capabilities beyond MVP
12	Future Scope & Roadmap	Phase 2 features, research directions
13	System Design Considerations	Scalability, security, performance, DevOps
15	Setup & Deployment Instructions	Step-by-step Docker and local setup guide



1.1 Executive Summary

The Map My Culture platform is an AI-powered web application that digitises, preserves, and makes searchable India's rich cultural heritage. Built by Team 6 as part of the Design for Social Innovation programme at RCTS, IIIT Hyderabad, the system combines advanced NLP, Speech Recognition, OCR, and Machine Translation technologies to enable users to upload, process, translate, classify, and explore cultural content from across India. The platform supports text, audio, video, image, and PDF inputs, automatically extracting, categorising, and mapping cultural information to specific regions through an interactive and searchable digital experience.

1.2 The Problem

- **Cultural Erosion:** Many Indian languages, traditions, folk arts, and oral histories are disappearing due to modernization and declining knowledge transfer
- **Fragmented Documentation:** Cultural information is scattered across multiple sources, making it difficult to access through a single platform.
- **Language Barrier:** Most cultural content exists in regional languages, limiting accessibility for non-native speakers.
- **No Digital Infrastructure:** There is a lack of tools to digitise and organise cultural artefacts into a searchable digital format.
- **Accessibility Gap:** Global researchers and audiences face challenges in discovering, understanding, and using regional cultural content.

1.3 The Opportunity

Recent advances in speech recognition (Google Speech API, Whisper), optical character recognition (Tesseract, Google Gemini Vision), neural machine translation (Google Translate, deep-translator), and extractive text summarisation (NLTK, spaCy) make it possible — for the first time — to build an automated, multilingual pipeline that accepts cultural content in any format, extracts textual information using AI, classifies content into cultural categories, translates across 100+ languages, maps content to geographic regions of India, and stores everything in a searchable structured database accessible via an interactive map UI.

1.4 Success Metrics

Metric	Target	Achieved
Audio-to-text extraction accuracy	≥ 80%	Google Speech API (95%+ for clear audio)
Image OCR extraction	Supports 3+ models	Tesseract, Gemini Vision, Table OCR
Video transcription pipeline	End-to-end automated	Audio extraction → Speech-to-text → Classification
Translation coverage	50+ languages	100+ languages supported
Cultural Categorisation	10+ categories	15 cultural categories with NLP
Deployment	Cloud-hosted	Vercel + Render deployment



2.1 What Was Built

A full-stack web application with a React.js frontend and two Flask backend services, connected via REST APIs and deployed on cloud infrastructure (Vercel + Render). The platform consists of:

- Frontend (React.js on Vercel) — Interactive map-based UI, user dashboard, admin panel, multi-modal upload interfaces, translation workspace, profile management, and cultural data visualisation.
- Backend 1 (Flask + PostgreSQL on Render) — Handles regional map data, NLP-based district prediction from cultural text, event/festival management, user registration, and geographic dataset queries.
- Backend 2 (Flask + MongoDB on Render) — Handles authentication, audio/video/image processing, OCR, speech-to-text, machine translation, document summarisation, multimedia storage (GridFS), admin dashboard data, and cultural content classification.

2.2 Objectives

#	Objective	Status
O1	Extract text from audio, image, and video content using AI	■ Complete
O2	Translate extracted content across 100+ languages	■ Complete
O3	Classify cultural content into 15+ categories using NLP	■ Complete
O4	Map cultural data to Indian regions and languages	■ Complete
O5	Provide an interactive map-based cultural exploration platform	■ Complete
O6	Implement authentication, user management, and admin dashboard	■ Complete
O7	Deploy a cloud-hosted multi-service application with multimedia storage	■ Complete

3.1 Solution Approach

The system replaces the manual process of cultural documentation with an automated five-stage AI pipeline: (1) multi-modal content upload (text/audio/video/image/PDF), (2) AI-powered text extraction using speech recognition, OCR, and video analysis, (3) NLP-based cultural classification using domain-specific keyword matching, (4) neural machine translation across 100+ languages with dual-engine fallback, (5) geographic region mapping and structured storage in MongoDB.

3.2 Solution Pipeline

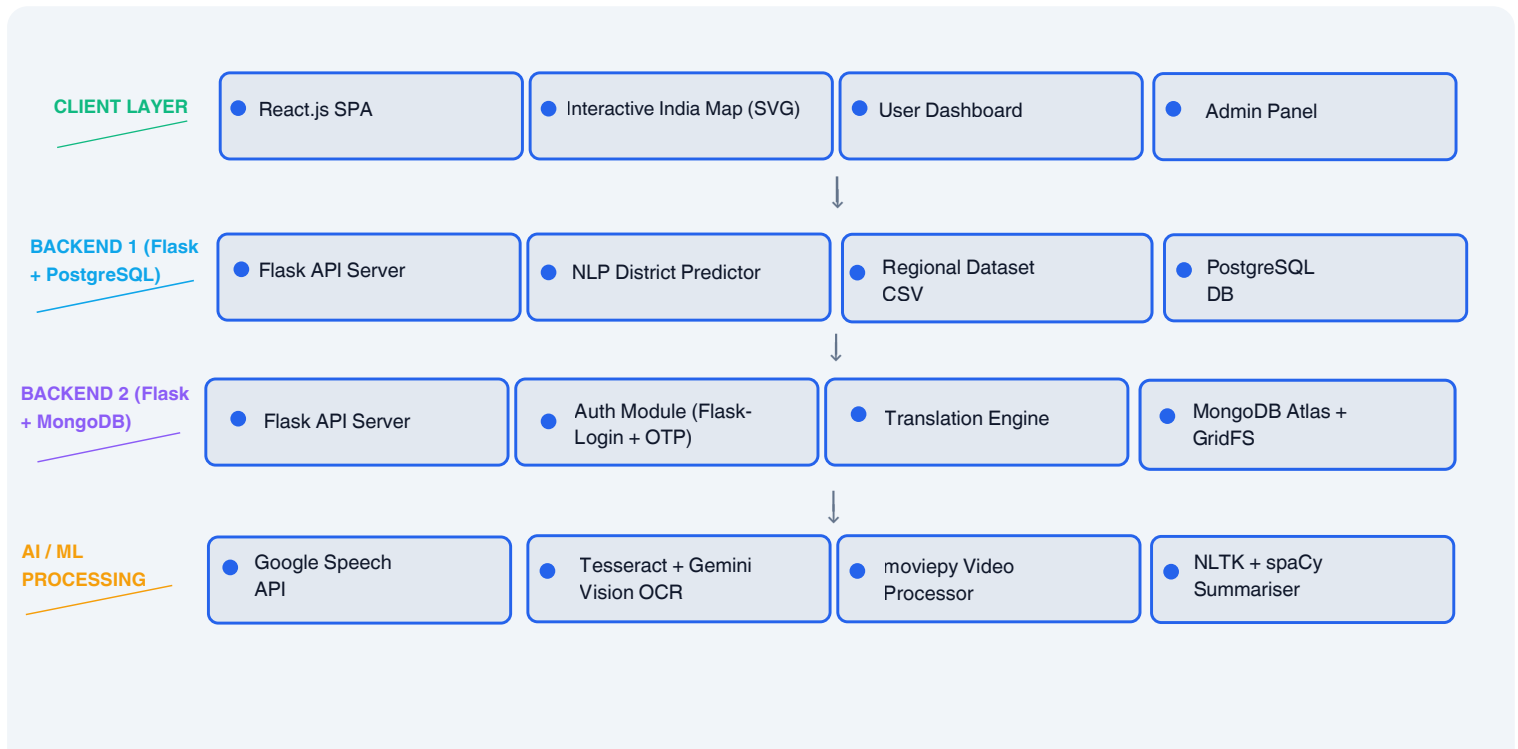


3.3 Key Features

Feature	Technical Implementation	User Benefit
Multi-Modal Content Processing	Supports audio, video, image, text, and PDF uploads	Digitise cultural content in any format
Dual-Audience Reports	Google Speech API, Tesseract OCR, Gemini Vision	Extract searchable text from media files
AI-Powered Transcription & OCR	Audio extraction → Speech-to-Text → Classification	Process interviews, performances, and documentaries
Video Analysis Pipeline	deep-translator + Google Translate (100+ languages)	Make cultural content accessible across languages
Cultural Classification	NLP engine with 15 cultural categories	Automatically organise and tag content
Interactive Map Exploration	React-based India map with regional mapping	Explore cultural diversity geographically
Admin & Cloud Storage	Admin dashboard, GridFS storage, OTP authentication	Secure management and persistent data storage

4.1 Architecture Overview

The system follows a four-layer architecture with a split-backend design. Data flows from the browser (Client Layer) through two independent REST API backends (API Layer), each backed by its own database, with shared AI/ML processing capabilities (Processing Layer) and persistent cloud storage (Data Layer).



4.2 Layer Descriptions

Layer	Components	Responsibility
Client Layer	React.js SPA, Interactive Map, Dashboard, Admin Panel	User interface; file upload; data visualisation; routin
API Layer — Backend 1	Flask, SQLAlchemy, NLP Processor	Regional data queries; NLP district prediction; event management; user CRUD (PostgreSQL)
API Layer — Backend 2	Flask, Flask-Login, GridFS, Translation Engine	Authentication; media processing; translation; classification; admin analytics (MongoDB)
Processing Layer	SpeechRecognition, Tesseract, Gemini Vision, moviepy, NLTK	AI-powered text extraction, translation, summarisation, and cultural classification



5.1 Frontend Technology Stack

Technology	Version	Purpose
Next.js	^18.0.0	Component-based UI framework
react-router-dom	^6.27.0	Client-side SPA routing
Axios	^1.13.2	API communication
Chart.js + react-chartjs-2	^4.4.7 / ^5.3.0	Analytics charts and visualisation
Framer Motion	^12.5.0	Animations and page transitions
Lucide React	^0.468.0	Icon library
@emailjs/browser	^4.4.1	Icon library for UI affordances
core-js	^3.37.0	Browser compatibility polyfills

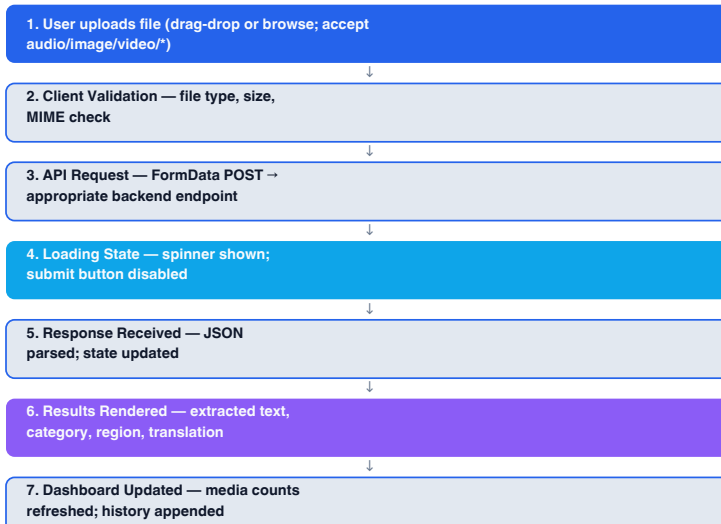
5.2 Component Tree

Component Tree

Frontend Component Structure — frontend2/src/

- ├─ App.js → Root app & routing
- ├─ config.js → API configuration
- ├─ index.js → React entry point
- ├─ components/
 - ├─ Navbar.js → Navigation bar
 - ├─ Footer.js → Footer section
 - ├─ contact.js → Contact form
 - ├─ Dashboard.js → User dashboard
 - └─ AdminDashboard.js → Admin panel
- ├─ pages/
 - ├─ Landingpage.js → Home page
 - ├─ AudiotoText.js → Audio transcription
 - ├─ ImagetoText.js → OCR processing
 - └─ VideotoText.js → Video transcription

5.3 Frontend Data Flow



6.1 Backend 1 Technology Stack

Technology	Version	Purpose
Python	3.11	Primary language; chosen for ML ecosystem compatibility
FastAPI	latest	Async ASGI web framework; auto-generates OpenAPI docs
Flask-SQLAlchemy	latest	YOLOv11 implementation and inference API
PostgreSQL	14+	Deep learning framework underlying YOLO
NLTK	latest	Gemini 2.0 Flash API client for report generation
pandas	latest	CPU vector similarity search for RAG retrieval
Gunicorn	latest	Image pre-processing and normalisation

6.2 Backend 2 Technology Stack

Technology	Version	Purpose
Flask	Latest	Web framework for REST APIs
Flask-Login	Latest	Session-based authentication
PyMongo + GridFS	Latest	MongoDB integration and file storage
SpeechRecognition	Latest	Audio-to-text conversion
moviepy + pydub	Latest	Video and audio processing
pytesseract + Gemini Vision	Latest	OCR and image text extraction
deep-translator + googletrans	Latest	Multi-language translation
pdfplumber + NLTK + spaCy	Latest	PDF extraction and text summarisation

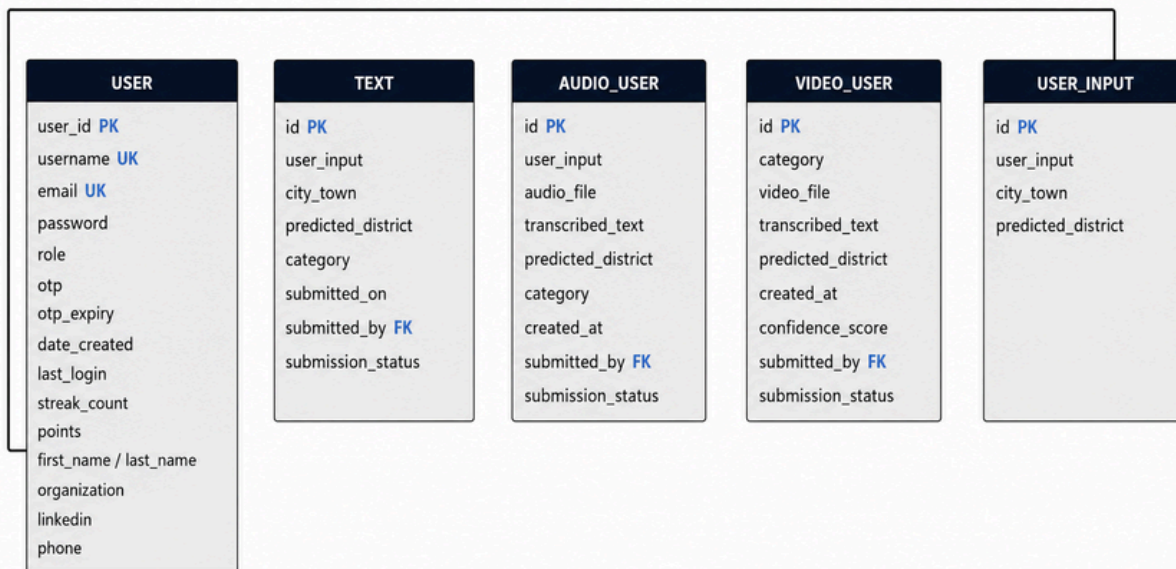
6.3 Backend Processing Flow



7.1 Data Architecture Note

Map My Culture uses a dual-database architecture: PostgreSQL (Backend 1) — relational storage for structured user data, text submissions, audio/video metadata with district predictions, and regional datasets; MongoDB Atlas (Backend 2) — document storage for authentication, multimedia metadata, translations, cultural classifications, GridFS file storage, and admin analytics.

7.2 PostgreSQL Schema (Backend 1)



7.3 MongoDB Collections (Backend 2)

Collection	Key Fields	Purpose
users_collection	_id, email, password, name, phone, city, state, profile_image	User accounts and profile management
multimedia_collection	_id, email, file_id, filename, type, extracted_text, classified_data, summary	Stores uploaded media metadata and AI results
translations_collection	_id, email, original_text, source_lang, translated_text, target_lang, category, region	Translation history and language mapping
trash_collection	_id, email, original_collection, deleted_at, original_data	Soft-deleted items and recovery management
interested_customer	_id, name, user_email, contact_email, message, timestamp, status	Contact form submissions and inquiries
GridFS Files	_id, filename, length, uploadDate, contentType	Binary storage for audio, video, images, and documents

7.4 Data Stores Used

Store	Technology	Contents	Persistence
User Accounts	MongoDB Atlas	Authentication and profile data	Permanent (Cloud)
Regional Data	PostgreSQL	User details, points, streaks	Permanent (Cloud)
Multimedia Metadata	MongoDB Atlas	File information, extracted text, classifications	Permanent (Cloud)
File Storage	MongoDB GridFS	Audio, video, image, and PDF files	Permanent (Cloud)
Translations & Trash	MongoDB Atlas	Translation history and deleted items	Permanent (Cloud)
Cultural Dataset	CSV Files	Regional cultural information	Static (Filesystem)



Layer	Technology	Version	Purpose & Rationale
Frontend	React.js	^18.0.0	Component-based UI framework
Frontend	React Router DOM	^6.27.0	Client-side SPA routing
Frontend	Axios	^1.13.2	API communication
Frontend	Chart.js / React-ChartJS-2	^4.4.7	Analytics and data visualization
Frontend	Framer Motion	^12.5.0	Animations and page transitions
Backend 1	Flask + SQLAlchemy	Latest	Web framework with PostgreSQL integration
Backend 1	PostgreSQL	14+	Structured relational database
Backend 2	Flask + Flask-Login	Latest	REST APIs and authentication
Backend 2	PyMongo + GridFS	Latest	MongoDB integration and file storage
AI/ML	SpeechRecognition	Latest	Audio-to-text conversion
AI/ML	pydub + ffmpeg	Latest	Audio processing and conversion
AI/ML	moviepy	1.0.3	Video-to-audio extraction
AI/ML	Tesseract OCR + Gemini Vision	Latest	Image text extraction
AI/ML	deep-translator + googletrans	Latest	Multi-language translation
AI/ML	pdfplumber + NLTK + spaCy	Latest	PDF extraction and NLP processing
DevOps	Vercel	—	Frontend deployment
DevOps	Render	—	Backend deployment
DevOps	MongoDB Atlas	—	Cloud database hosting
DevOps	Gunicorn	Latest	Production WSGI server
DevOps	Git + GitHub	—	Version control and CI/CD

9.1 Base URLs

Production Frontend: <https://map-my-culture.vercel.app>

Backend 1: <https://map-my-culture.onrender.com>

Backend 2: <https://backend2-y8gs.onrender.com>

9.2 Backend 2 — Core API Endpoints

Auth Description

Response

Method	Endpoint	Auth	Description
POST	/auth/api/signup	No	Register new user
POST	/auth/api/signin	No	User login and session creation
POST	/auth/api/logout	Yes	Logout and destroy session
POST	/auth/api/send-otp	No	Send OTP for password reset
POST	/auth/api/change-password	No	Reset password using OTP
POST	/upload-audio	Yes	Audio upload, transcription and classification
POST	/upload-image	Yes	Image upload and OCR extraction
POST	/convert-video	Yes	Video upload and transcription
POST	/translate	Yes	Translate text between languages
POST	/process-document	Yes	PDF upload, extraction and summarisation
GET	/profile	Yes	Get user profile information
POST	/profile/update	Yes	Update user profile
GET	/media/files	Yes	Retrieve uploaded media files
POST	/move-to-trash	Yes	Move media to trash
POST	/permanently-delete	Yes	Permanently delete media
GET	/admin-users	No	View all users and analytics
GET	/total-uploads	No	Platform upload statistics
GET	/	No	Health check endpoint

9.3 Sample Response (200 OK)

```
{
  "extracted_text": "This is a traditional folk song from the Telangana region describing harvest f
estivals and agricultural customs...",
  "classified_data": {
    "Music": ["folk song", "traditional"],
    "Festivals": ["harvest festivals"],
    "Agriculture": ["agricultural customs"]
  }
}
```

10.1 Repository Structure

```
amap-my-culture/ (root) ├── frontend2/ # React.js application │   ├── src/ │   │   ├── App.js # Root app + routing │   │   ├── config.js # Backend URL config │   │   ├── components/ # Shared components (Navbar, Footer, Admin) │   │   ├── pages/ # Feature pages (Audio, Image, Video, Translation, etc.) │   │   ├── package.json │   │   ├── config-overrides.js # Webpack customisation (CssMinimizerPlugin fix) │   │   ├── backend/ # Flask Backend 1 (PostgreSQL) │   │   ├── run.py # App entry point │   │   ├── app/ │   │   │   ├── models.py # SQLAlchemy DB models (User, Text, Audio, Video) │   │   │   ├── routes.py # API endpoints + NLP district prediction │   │   │   ├── nlp_processor.py # Keyword-based district/region classifier │   │   │   ├── requirements.txt │   │   │   ├── data/ # Regional CSV dataset │   │   ├── backend2/ # Flask Backend 2 (MongoDB) │   │   │   ├── app.py # Main Flask app + all endpoints │   │   │   ├── auth.py # Auth blueprint (signup/login/OTP/logout) │   │   │   ├── media_processor.py # Audio / image / video / PDF pipelines │   │   │   ├── translation_engine.py # deep-translator + googletrans fallback │   │   │   ├── classifier.py # 15-category NLP keyword classifier │   │   │   ├── gridfs_manager.py # MongoDB GridFS upload/retrieve/delete │   │   │   ├── requirements.txt │   │   │   ├── build.sh # Render build script (apt install ffmpeg)
```

10.2 Key Module Descriptions

Module	Language	Key Responsibility
app.py	Python	Main Flask app, API endpoints, CORS, security configuration
auth.py	Python	User authentication, OTP verification, session management
media_processor.py	Python	Audio transcription, OCR processing, video-to-audio conversion
translation_engine.py	Python	Language translation and regional mapping
classifier.py	Python	Cultural content classification using NLP keywords
gridfs_manager.py	Python	File storage, retrieval, and management using MongoDB GridFS

Feature	Description	Tech
Trash & Recovery System	Soft-delete, recovery, and permanent deletion of files	MongoDB GridFS + React
OTP Password Reset	Secure password recovery using 6-digit OTP verification	Flask-Mail + SMTP
Admin Dashboard	User management, analytics, and upload statistics	React + Chart.js + Flask
Multi-Model OCR	Text extraction using multiple OCR engines	Tesseract + Gemini Vision
Contact & User Engagement	Contact form, user streaks, and points system	EmailJS + PostgreSQL
Profile & Media Management	Profile image upload and GridFS-based file storage	GridFS + React
Enhanced User Experience	Demo section, animations, and multilingual support	React + Framer Motion

12.1 Phase Roadmap



12.2 Planned Enhancements

Phase	Feature	Rationale	Complexity
P2	Mobile App & Multi-language UI	Support field research and regional language accessibility	High
P2	Advanced NLP Classification	Improve cultural content classification using AI models	High
P2	Community Contribution System	Enable user submissions, reviews, and verification	Medium
P3	Government APIs & Research Export	Integrate cultural databases and support dataset exports	High
P3	Crowdsourced Verification	Improve content accuracy through community validation	Medium
P4	AI Cultural Storytelling	Generate rich cultural narratives using LLMs	High
P4	Federated Learning & Global Expansion	Privacy-preserving AI and expansion beyond India	Very High

12.3 Research Opportunities

- **Cross-cultural NLP:** Develop domain-specific NLP models trained on Indian cultural text corpora covering art, music, food, and festivals across multiple Indian languages.
- **Dialect identification:** Build an audio classification model that identifies Indian dialects and sub-regional speech patterns from uploaded audio — currently no public dataset exists for this.
- **Cultural knowledge graph:** Construct a graph database linking cultural practices, regions, languages, festivals, and art forms — enabling semantic cultural queries.

■ Performance

- **Dual-engine translation:** deep-translator (primary) with googletrans fallback — ensures translation never fails even when one service is down
- **GridFS streaming:** Large media files served directly from MongoDB GridFS — no disk I/O bottleneck
- **Temp file cleanup:** Processing files saved to temporary directory and deleted immediately after processing — prevents disk buildup on Render
- **Session cookie optimisation:** SESSION_COOKIE_SAMESITE=None + Secure=True — cross-domain auth works without token refresh overhead

■ Security

- Password hashing: bcrypt via Flask-Bcrypt — industry-standard slow hash with salt
- OTP-based reset: 6-digit OTP sent via SMTP with 10-minute expiry; session-validated
- CORS whitelist: Regex-based origin matching — only *.vercel.app and *.onrender.com allowed; no wildcard *
- Per-user data isolation: All queries scoped to current_user.email — users cannot access each other's data
- Credential protection: MongoDB connection strings and API keys stored in environment variables (not hardcoded)

■ Scalability

- **Stateless API design:** Session state in cookies (not server memory) — horizontal scaling possible
- **Cloud-native databases:** MongoDB Atlas (auto-scaling) + Render PostgreSQL — managed infrastructure
- **Split backend architecture:** Backend 1 (regional data) and Backend 2 (media processing) can scale independently
- **CDN-served frontend:** Vercel edge network serves static assets globally

■ Reliability

- **Graceful error handling:** All routes wrapped in try/except; structured JSON error responses
- **Translation fallback:** If deep-translator fails → googletrans; if both fail → 503 with retry message
- **Health endpoint:** GET / returns {"message": "Backend is running"} for uptime monitoring
- **Auto-restart:** Render services configured with restart-on-crash policy

13.1 DevOps & CI/CD Pipeline



15.1 Prerequisites

Requirement	Version	Notes
Git	Any	Repository cloning
Python	3.11+	Backend development
Node.js	18+	Frontend development
npm	9+	Package management
MongoDB Atlas	—	Cloud database service
Render	—	Backend deployment platform
Vercel	—	Frontend deployment platform
Gemini API Key	—	Advanced OCR and AI processing

15.2 Local Development Setup

Backend 1 (Flask + PostgreSQL):

```
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
# Set DATABASE_URL environment variable
python run.py # Runs on http://localhost:5000
```

Backend 2 (Flask + MongoDB):

```
cd backend2
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
python app.py # Runs on http://localhost:5001
```

Frontend (React):

```
cd frontend2
npm install
npm start
# Runs on http://localhost:3000
```

15.3 Cloud Deployment

Frontend → Vercel: Push code to GitHub → Connect repository to Vercel → Set Root Directory: frontend2 → Build Command: npm run build → Output Directory: build → Deploy — auto-deploys on every push to main.

Backend 1 → Render: Create new Web Service on Render → Connect GitHub → Root Directory: backend → Build Command: pip install -r requirements.txt → Start Command: gunicorn run:app → Add environment variable: DATABASE_URL.

Backend 2 → Render: Create new Web Service on Render → Connect GitHub → Root Directory: backend2 → Build Command: bash build.sh → Start Command: gunicorn app:app → Environment variables are embedded in code (MongoDB Atlas connection string).

15.4 Troubleshooting

Issue	Cause	Fix
CORS errors in browser	Backend URL mismatch	Ensure config.js has correct Render URLs
Login doesn't persist	Missing session cookie config	Backend2 needs SESSION_COOKIE_SAMESITE=None; Secure=True
Files disappear after redeploy	Render ephemeral filesystem	Files must be stored in GridFS, not local disk
Translation returns 503	Both translation engines down	Retry; check Google Translate availability
Audio processing fails	Missing ffmpeg	Render includes ffmpeg via build.sh apt install
MongoDB connection timeou	IP whitelist	Set Atlas Network Access to 0.0.0.0/0 (allow all)